CORS

R4.real.c10

Cross-Origin Resource Sharing

Définition : un mécanisme qui permet à des ressources restreintes d'une page web d'être récupérées par un autre domaine extérieur au domaine à partir duquel la première ressource a été servie.

Sans CORS, je charge une page web sur un domaine, la page charge un JS et une CSS, la CSS charge des fonts sur un autre domaine, et le Js import des modules du puis un autre domaine, les modules réalisent des requêtes AJAX sur encore un autre domaine => une multiplicité d'accès => des failles de sécurité

https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

Exemple

domain.com dispose de /api/v0/account qui renvoie des données sensibles

Un script malveillant sur un autre domaine qui lorsqu'un utilisateur de la plateforme visitera le site hébergeant le code malveillant, il fera une requête vers « https://domain.com/api/v0/account » et retournera la réponse sur le serveur de l'attaquant.

L'importance de la SOP réside dans sa capacité à limiter l'impact d'attaques de type cross-site scripting (XSS) et à contrer l'accès en lecture à d'autres sites (donc prévenir le vol de données entre sites).

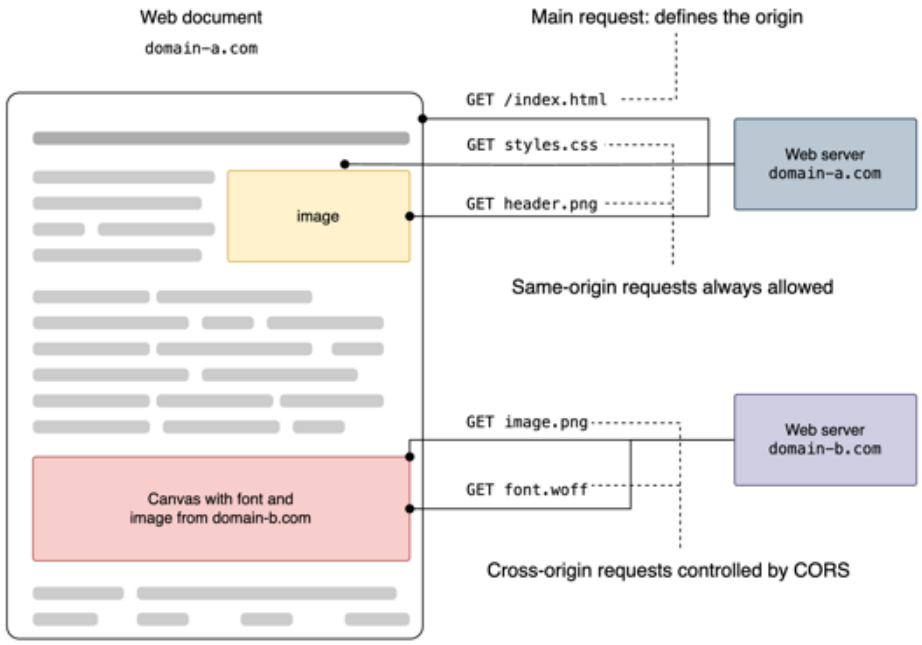
Same-Origin Policy

URL	Résultat	Raison
http://example.com/*	OK	Seul le chemin diffère
https://example.com/*	KO	Pas le même protocole
http://example.com:1234/*	KO	Pas le même port

```
http://example.com/
```

=> Trop restrictif par exemple pour l'utilisation d'une API publique

La première requête définie l'origine



En-têtes

Requêtes

- Origin
- Access-Control-Request-Method
- Access-Control-Request-Headers

Réponse

Access-Control-Allow-Origin

Access-Control-Allow-Credentials

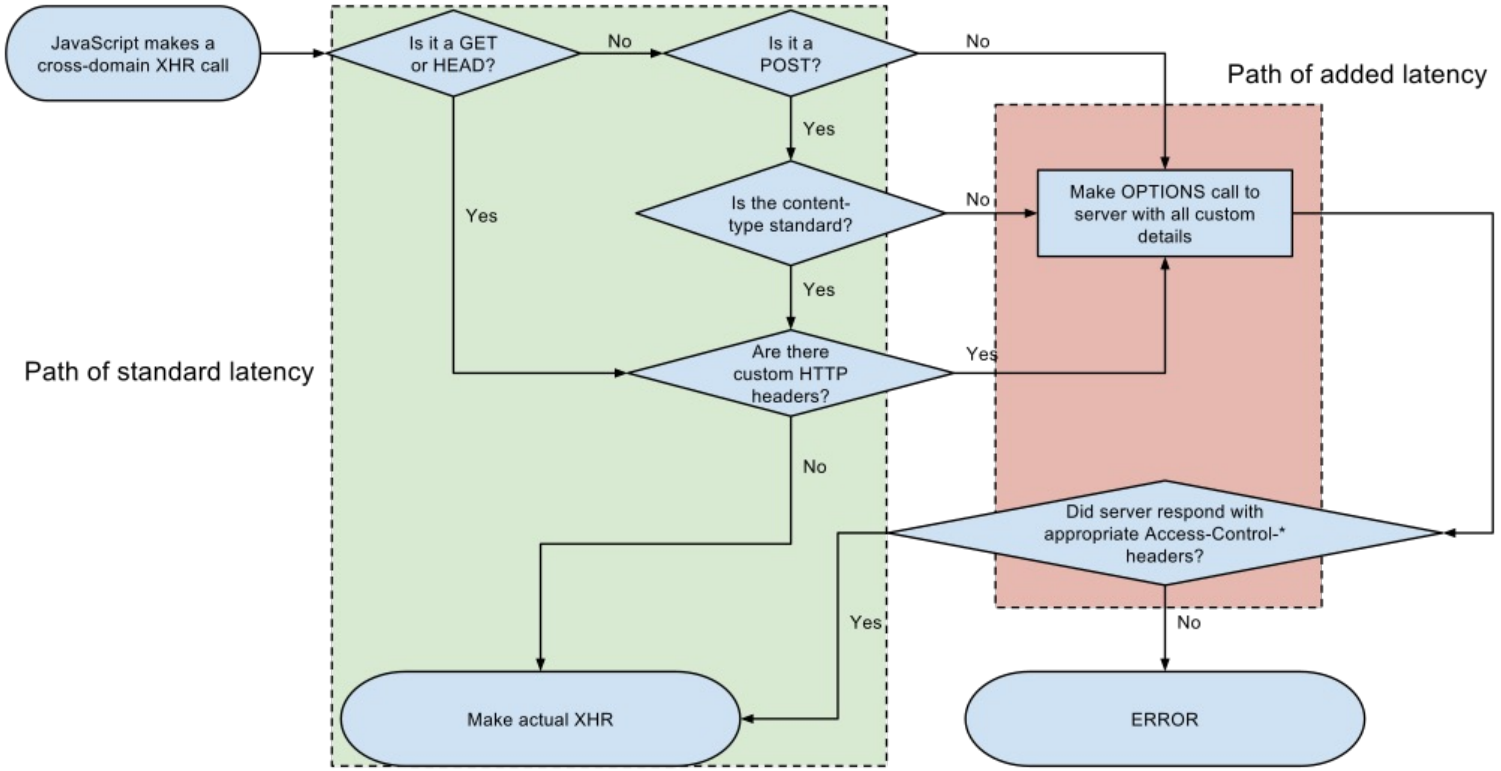
Access-Control-Expose-Headers

Access-Control-Max-Age

Access-Control-Allow-Methods

Access-Control-Allow-Headers

Algorithme avant un fetch



Exemple de requête avec contrôle amont

Le client : demande des information au serveur avant de charger le premier document.

OPTIONS / Host: service.example.com

Origin: http://www.example.com

Les serveur répond :

Access-Control-Allow-Origin: http://www.example.com Access-Control-Allow-Methods: PUT, DELETE

Les prochaines requêtes ne pourront être que sur exemple.com en http, sur le port 80 avec les méthode PUT et DELETE

Access-Control-Allow-Origin: *

La valeur « * » est spéciale en ce sens qu'elle ne permet pas aux requêtes de fournir des informations d'identification, c'est-à-dire n'autorise pas l'authentification HTTP, les certificats SSL côté client, ni l'envoi de cookies dans la requête inter-domaines

Problème

https://www.vaadata.com/blog/fr/comprendre-et-prevenir-les-problemes-de-configuration-de-cors/

Origine reflétée

Pour ne pas maintenir une liste le serveur refléte l'origine

```
Serveur hébergé sur le domaine « domain.com » avec un
endpoint tel que « /api/v1/account/ » qui renvoie des
données sensibles d'utilisateurs d'une application web.
```

Problème

GET /api/v1/account/ HTTP/2 Host: domain.com

Cookie: session=v77QFGNrzt4upa4W0SuxLFMCOKmEibNo

Accept-Encoding: gzip, deflate

Origin: **https://test.com**

L'origine a été modifiée

HTTP/2 200 OK

Access-Control-Allow-Origin: https://test.com

Access-Control-Allow-Credentials: true

Content-Type: application/json; charset=utf-8

X-Frame-Options: SAMEORIGIN
Content-Length: 149

{"données sensibles":"données sensibles"}

Problème

- vérification basée sur une expression régulière
  .+domain\.com => le point n'est pas escapé
  bypassdomain.com

- Origine null autorisé

GET /api/v1/account/

Host: domain.com

Origin: null

HTTP/1.1 200 OK

Access-Control-Allow-Origin: null

Access-Control-Allow-Credentials: true

<iframe sandbox="allow-scripts allow-top-navigation allow-forms" src="data:text/html,<script>[Code javascript]</script>"></iframe>

Pour pouvoir faire des requêtes depuis une origine null, on peut utiliser la balise script combinée à l'attribut « sandbox ».

…

Sous domaines

…

Bonne pratique

liste blanche sur des domaines de confiance

ou privilégier une authentification à partir d’en-tête

Bien, mal, utile ?

```
app.use((req,res,next)=>{
    res.setHeader("Access-Control-Allow-Origin",'*');
    res.setHeader("Access-Control-Allow-Methods",'OPTIONS, GET,
POST, PUT, PATCH, DELETE');
    res.setHeader("Access-Control-Allow-Headers",'Content-
Type,Authorization');
    next();
})
```